循环读出压缩：一种自适应的视觉语义压缩方法

XX[1)]

[1)]（xxx，xxx 518000）

摘 要 多模态大模型以稠密的图像块 token 作为视觉接口，token 数随分辨率上升到数百乃至上千；在对上下文长度与实时性敏感的应用中，上下文很快被大量低信息量的 token 占满。把图像压缩为固定数量的 token 是常见做法，但压缩预算在训练完成后固定，高精度场景会因此丢失细节。基于此，本文基于 looped transformer 的思想提出循环读出压缩（recurrent read-out compression），具体为：编码器在输入序列中拼接 K 个可学习特殊 token，训练时，解码器只能从这 K 个 token 获取图像信息，并据此重建整幅图像。解码器以权重共享的 Transformer 块反复读出这些 token。在 ImageNet-100 上的受控实验表明：在 240M 档（编码器与解码器各约 240 M、合计 481.54 M）下，以 token 数计的压缩率由旧 63M 基准的 3.06 倍（196→64）提高到 6.12 倍（196→32），单步读出达到 32.35 dB，显著高于逐图像块直流基线（18.45 dB）；把同一模型展开到 16 步，PSNR 由 22.07 dB 升到 27.94 dB。与逐块 PCA 的对照按每图连续值个数分层：在 $\leq$ 3,584 个连续值的紧预算端，本文两种瓶颈码（每 token 448 维与 224 维）的 PSNR 均高于同值数 PCA（+0.89 $\sim$ +1.41 dB）；7,168 值处 448 维码仍高 0.34 dB（224 维码仅 L1 更低）；预算放宽到 14,336 值及以上时 PCA 反超（+1.37 dB；主模型在 39,936 值处为 27.94 dB 对 35.75 dB）。把每个特殊 token 由 1248 维压到 448 维（每图连续值降为 36%）只损失 0.25 dB，在同一 14,336 值下比旧 63M 基准高 3.54 dB。在公开人评复杂度数据上，图像复杂度与重建误差正相关（ρ 在 5 个容量档 $\times$ 3 个子集共 15 个组合上全部为正，范围 +0.33 $\sim$ +0.54），容量增加缩小的是复杂度惩罚的绝对幅度；容量阶梯在 240M 档达到峰值。总体而言，循环读出压缩以一组固定的 token 配合自适应的读出，在单一模型中提供由少到多的多档 token 预算。

关键词 视觉 token 压缩；循环读出；权重共享；特殊 token；深监督；注意力机制
中图法分类号 TP391 **文献标识码** A

Recurrent Read-Out Compression for Adaptive Visual Token Compression

xxx[1)]

[1)](xxx, x, x, x 518000, xx)

Abstract Multimodal large models interface with images through dense patch tokens, whose count grows with resolution to hundreds or even thousands; in applications sensitive to context length and latency, such as embodied agents, the context is quickly exhausted by many low-information tokens. Compressing an image into a fixed number of tokens is common practice, but the budget is frozen once training ends, so high-precision scenes lose detail. We propose recurrent read-out compression: the encoder appends K learnable special tokens to its input sequence; during training, the decoder can obtain image information only from these K tokens and must reconstruct the whole image from them, so the tokens must jointly contain all the information needed for reconstruction rather than each repeating the same global information. Whether the compression is effective depends on whether the information is preserved, not on the token count: layout, objects and boundaries remain recoverable after compression. The decoder repeatedly reads the tokens out

with a weight-tied Transformer block: the first-step query is a learnable query base, the query at step t is the query base plus the previous output, and step t reads only its own read-out window; every step predicts the full image and receives pixel-level supervision. The token budget is thereby tied to the number of read-out steps: the same weights can be stopped at any step at inference, so the budget is no longer fixed at training time but can be chosen at inference according to the precision a scene requires. The design borrows the capacity-increasing allocation of adaptive-length tokenization, but the loop acts on the read-out side. Controlled experiments on ImageNet-100 show that at the 240M scale (about 240M parameters each for encoder and decoder, 481.54M in total) token compression is raised from 3.06× (196→64) in the earlier 63M baseline to 6.12× (196→32), and single-step read-out reaches 32.35 dB, far above the per-patch DC baseline (18.45 dB); expanding the same model to sixteen steps raises PSNR from 22.07 dB to 27.94 dB. Against a per-patch PCA baseline at matched numbers of continuous values the outcome is budget-dependent: at the tight end (up to 3,584 values per image) both of our bottleneck codes (448- and 224-dimensional tokens) are +0.89 to +1.41 dB above PCA at the same value count; at 7,168 values the 448-dimensional code is still 0.34 dB higher while the 224-dimensional one only wins on L1; from 14,336 values on PCA takes the lead (+1.37 dB; at 39,936 values the main model gives 27.94 dB against 35.75 dB), because PCA keeps all 196 spatial positions and only reduces the dimension at each of them, whereas we route the whole image into 32 tokens. Squeezing each token from 1248 to 448 dimensions (36% of the continuous values) costs only 0.25 dB and remains 3.54 dB above the earlier 63M baseline at the same 14,336 values. On public human-rated complexity data, complexity correlates positively with reconstruction error (positive for all 15 capacity×subset combinations, ranging from +0.33 to +0.54), and increasing capacity shrinks the absolute complexity penalty; the capacity ladder peaks at 240M. Overall, recurrent read-out compression pairs a fixed token set with adaptive read-out, offering a family of token budgets from few to many within a single model.

Keywords visual token compression; recurrent read-out; weight tying; special tokens; deep supervision; attention mechanism

1 引言

视觉—语言模型（vision–language model, VLM）通常把图像切分为 16×16 或 14×14 的图像块（patch），逐块投影为稠密视觉 token，再与文本 token 拼接后送入语言模型[1-5]。这一接口简单有效，但代价随分辨率迅速上升：224×224 的输入在 patch 为 16 时产生 $N = 196$ 个视觉 token，448×252 的输入在 patch 为 14 时产生 $N = 576$ 个视觉 token。相邻 patch 携带的信息高度重叠，而语言模型的注意力开销随视觉 token 数增长，上下文预算被大量低信息量 token 占用。在对话、视频问答等长上下文场景中，这一矛盾尤其突出[6-7]：瓶颈往往不是“图像能否放进上下文”，而是“放进去之后还剩下多少信息”。

视觉 token 压缩是缓解这一矛盾的主要途径。一类做法在编码器之后接一个独立的压缩模块，把稠密特征映射为少量 token[8,3,9-10]；另一类做法在编码器内部或输入端引入可学习的特殊 token，让编码器自行把图像信息汇聚到少量 token 上[11-13]。前者把“表示”与“压缩”分成两步，编码器本身没有产生紧凑表示的压力；后者更接近本文的出发点。两类做法通常把一个模型与一个长度绑定：模型训练完成后，预算不能再随场景调整；要在多个预算下取用，就需要为每个预算单独训练或重新编

码。

压缩后的视觉向量本身已经足以支撑下游任务。Q-Former 用少量可学习查询把图像压成固定数量的 token 供语言模型使用[3]；LLaMA-VID 每帧只用两个 token（上下文 token 与内容 token），即可支撑小时级视频的对话与问答[6]；LLaVA-Mini 把每张图压到 1 个 token，在 11 个图像基准与 7 个视频基准上仍优于使用 576 个 token 的 LLaVA-v1.5[7]；VoCo-LLaMA 在指令微调阶段蒸馏语言模型对视觉 token 的理解，把压缩比推进到 576 倍[14]；Matryoshka 多模态模型把视觉内容表示为一组可按前缀取用的嵌套 token，发现 COCO 类基准只需约 9 个 token 就能接近全部 576 个 token 的准确率[15]；SEED 与 VILA-U 则用离散视觉 token 统一了理解与生成[16-17]；视觉投影器一侧也在处理同一问题，Honeybee 把“可灵活调整 token 数”与“保留局部上下文”列为一对核心要求[18]。这些结果说明，压缩后的表示用于对话等下游场景已不是问题，真正的问题是：压到多小之后信息还剩下多少，以及同一个模型能否在不同预算之间平稳切换。本文因此把像素重建当作“信息是否保留”的直接探针[19-20]，而不把重建本身当作最终目的。

让长度本身可变是摆脱固定预算的一条途径。自适应长度图像分词（ALIT）[19]是这一途径上与本文结构最接近的工作：它把长度的决定放进循环，以同一组蒸馏权重反复滚动，每轮将图像与既有隐 token 一并重新蒸馏，并追加新的可学习 token，在共享码本下产生一族长度不同的表示（32–256 个 token），并表明 token 数与图像熵、熟悉度以及下游任务需求相关。本文借鉴其中两个要素：同一组权重的重复施加，以及逐步递增的表示容量；但把位置换到读出侧。具体而言，本文把压缩写进编码器的训练目标：在编码器输入序列中拼接 K 个可学习特殊 token（special token），以“仅用这 K 个 token 还原整幅图像”作为训练信号。由于 $K \ll N$，逐 patch 的局部复制无法完成重建，编码器必须把跨区域的证据聚合到少量 token 上。

压缩完成之后，读出方式决定了一个模型能覆盖几个预算：若 K 个 token 被一次性全部读出，就只有训练时定下的那一个预算。ALIT 把预算的决定交给编码器，长度随图而定；本文把可变的量放在读出侧：K 在训练后固定，每一步允许解码器读入其中多少由步数决定。为此，本文把读出过程组织成**循环读出**：一个权重共享的 Transformer 解码块被反复施加，第 t 步只读取第 t 个读窗口内的特殊 token，第 t 步的查询由固定查询基与上一步输出相加得到；每一步都直接预测整幅图像并接受监督。随着步数增加，解码器读到的 token 数逐步增加，因此同一模型在推理时可停在任意一步，在不同 token 预算下复用。预算由此从训练期设定变为推理时的选择。本文把这种读出步数随给定预算调整的方式称为自适应读出（adaptive read-out）；自适应的是读出，而不是由模型按图决定预算。这一“预算可变”的思路在图像压缩中并不新：Toderici 等用共享参数的循环编解码器实现了一次训练、多码率部署[21]。本文关心的不是“能否让预算可变”，而是把自适应读出放到视觉 token 接口上之后，它换来了什么、又要付出什么。

本文的主要贡献如下。

(1) **读出侧自适应的 anytime 码**：编码器一次前向把图像压缩为 K 个特殊 token；解码器以权重共享的 Transformer 块按读窗口逐步读出，每步直接预测整幅图像并接受像素监督，因此每个读出前缀都是一个合法码；停在任意一步的输出与完整前向的对应步逐位一致，同一模型无需重训或重新编码即可在不同预算下取用。与把预算写进码本身的嵌套式做法不同，这里改变的是读出程序执行到第几步，而不是码的前缀有多长。

(2) **自适应读出的代价与前沿**：自适应读出不是免费的。本文在受控条件下给出“终端质量—前缀可用性”的可测量前沿：逐步监督的权重调度偏向末步会提高末端质量，但会牺牲中间档位；等权监督保留全档可用性，却压低末端质

量。由于对照组共用同一结构与同一训练预算，这一前沿把“自适应读出的代价”从定性说法变成可比较的量，并给出一个可操作的工作点。

(3) **预算分配的收益上界**：本文进一步检验“把预算按图分配”是否值得。在逐图逐步的重建数据上，逐图最优停步相对单一固定预算的增益接近于零；在等平均 token 数下，逐图自动分配反而更差；即使允许逐图重新分配预算，其上界也远小于档位之间的差距。机制在于逐图误差排序在预算轴上几乎不变。因此自适应读出的价值在部署级预设档位；至于由模型按图自动决定预算，本文的结论是它的增益可以忽略。

2 相关工作

视觉 token 压缩与视觉分词. 视觉 token 压缩的目标是用少量 token 表示整幅图像。VQ-VAE[22]与 VQGAN[23]把图像编码到离散码本，使“用离散 token 表示图像”成为可训练的框架；TiTok[9]把图像压成 32 个一维离散 token 仍能完成重建与生成；MAGVIT-v2[10]则表明，分词器的质量是语言模型在视觉生成上超过扩散模型的关键。这条线索把压缩本身作为目标，通常配有量化与熵编码，评价标准是重建或生成质量；但同一个分词器只给出一种长度，预算在训练完成时即被固定。

压缩表示的下游使用. 压缩后的视觉向量能否被语言模型直接使用，已有大量正面证据。Q-Former 用少量可学习查询把图像压成固定数量的 token 交给语言模型[3]；LLaMA-VID 每帧只用上下文 token 与内容 token 两个 token，即可支撑小时级视频的对话与问答[6]；LLaVA-Mini 通过模态预融合把每张图压到 1 个 token，在图像与视频基准上仍优于使用 576 个 token 的 LLaVA-v1.5[7]；VoCo-LLaMA 在指令微调阶段把语言模型对视觉 token 的理解蒸馏到压缩 token 上，压缩比达到 576 倍[14]；Matryoshka 多模态模型把视觉内容表示为一组可按前缀取用的嵌套 token，支持推理时按实例选择粒度，并发现 COCO 类基准只需约 9 个 token 就能接近全部 576 个 token 的准确率[15]；SEED 与 VILA-U 用离散视觉 token 统一了理解与生成[16-17]；Honeybee 则把“可灵活调整 token 数”与“保留局部上下文”列为视觉投影器的一对核心要求[18]；自适应长度分词则从另一侧给出旁证：token 数应与图像熵、熟悉度及下游任务需求匹配，而不是固定不变[19]。这些工作说明压缩表示用于对话、问答与生成已经可行；同时也提示两点：其一，重建保真度是可用的必要条件而非充分条件，任务层面的最优粒度与重建指标并不一致[15]；其二，预算是否可变、在何时决定，仍是各方法各自的设计选择。本文的像素重建探针只用于判断信息是否保留，不据此宣称下游语义充分性。

可变长度分词与循环分配. 另一类工作使长度本身可变。嵌套 dropout[24]用随机保留前缀、丢弃其余单元的方式学习有序表示；Matryoshka 表示学习[25]要求特征向量的低维前缀即可独立完成任务；FlexTok[26]把图像重采样为可按前缀缩短到任意长度的一维 token 序列；ElasticTok[27]先学一个最长表示，再在推理时搜索它的子集。这些方法都在一次前向内决定长度，且码本身被训练成有序结构，缩短前缀仍然可用。ALIT[19]把这一决定放进循环：先用预训练 VQGAN 把 256×256 图像编码为 16×16 个二维 token，再蒸馏为 32–256 个一维隐 token；每轮滚动都把上一轮的隐 token 与新增的可学习 token 拼接后重新送入共享的蒸馏编码器—解码器，二维 token 一并被重新编码，并按上一轮的重建质量掩掉已充分蒸馏的部分；各轮共用同一个量化码本，于是长度成为同一容量族中的一个选择，可按重建误差阈值逐图选取。本文借鉴其中的权重共享循环与逐轮递增的容量，但将循环从编码侧的蒸馏移至解码侧的读出：容量增长不依赖新增 token，而由读窗口的宽度扩张给出；预算不由编码期逐图分配，而由推理期的停止点给出（逐项对照见第 5.3 节）。

编码器内部的可学习 token. 用一组可学习查询读取输入是压缩视觉特征的另一条主线。Perceiver IO[8]用固定数量的隐变量数组读取任意长

度的输入；BLIP-2 的 Q-Former[3]在冻结图像编码器之后用少量查询抽取信息；ViT 的 register token[11]通过在序列中额外加入不参与输出的 token 来改善注意力路由。更接近本文设定的是把可学习 token 直接放入编码器：FALCON[12]把视觉 register 放进视觉编码器内部，使编码器直接输出紧凑 token 集；另有工作在冻结 DINOv2 之上用少量可学习查询并辅以显式重建目标；ICAE[13]则在语言模型输入序列中插入可学习记忆槽。与上述工作相比，本文的特殊 token 参与编码器全部层的双向自注意力，且唯一的训练信号是像素重建；读出侧不是一次前向，而是权重共享的循环分块读出。

权重共享与循环计算. 重复施加同一组权重是获得迭代细化的经典手段。Universal Transformer[28]在层间共享权重并按需迭代；深度均衡模型[29]把前向过程视为求解不动点；Giannou 等[30]证明循环 Transformer 可以模拟程序，循环步充当工作记忆；Recurrent Interface Network[31]让图像 token 与隐 token 反复读写，把循环用于生成；Toderici 等[21]用共享参数的循环编解码器做图像压缩，在每次迭代上都施加损失，并通过时间反向传播训练，其部署时只需训练一次即可按迭代数得到不同码率，且“一次重建”（one-shot）变体在每步直接预测整幅图像。扩散模型[32]则用同一去噪网络在多个噪声层级上重复施加，天然形成由粗到细的过程。本文借鉴“同权重、多次施加”的结构，但把循环从“执行算法”或“去噪”移到“读出压缩表示”，并把中间各步的质量与读出步数可变的代价作为可检验的问题。

逐步监督与由粗到细解码. 深监督[33]在网络的多个中间层施加监督，使浅层也能获得有效梯度。在视觉生成中，MaskGIT[34]与 VAR[35]按由粗到细的尺度或掩码比例组织解码；D-AR[36]让不同位置的 token 对应不同的去噪步，从而无需改动训练或推理策略就得到一条由粗到细的 token 顺序。这些工作的顺序来自分词器或生成过程的先验设计。本文的顺序来自读出侧的分块计划：第 t 步读取第 t 个读窗口，窗口宽度随步数增长；每一步都直接预测整幅图像，因此逐步曲线本身构成一个可观测的细化过程。

本文定位与边界. 按“预算可变性发生在哪一侧”，上述工作可以排成一条轴：一侧把长度写进码本身（嵌套 dropout、Matryoshka、FlexTok、ElasticTok 与 ALIT 在编码或生成阶段决定长度，码因此有序或可按前缀取用）；另一侧只改变读出，码保持定长，读出步数在推理时按给定预算确定（本文）。这条轴区分的是“预算在何时、由谁决定”，而不是方法之间的优劣：ALIT 的按图定长与 Matryoshka 多模态模型的按实例选粒度都是各自有效的自适应方案，本文的取舍只是把可变性放到读出侧。本文不主张首次提出可变的预算——这一思路在图像压缩中已有先例（Toderici 等）——而是把自适应读出放到视觉 token 接口上，受控地测量其代价、前沿与预算分配的收益上界。相应地，本文只以像素重建为探针，不把结论外推到对话等下游性能；机制见第 3 节，受控实验见第 4 节，与 ALIT 的逐项对照见第 5.3 节。

3 方法

本节按数据流依次说明循环读出压缩的各个组成部分。

3.1 问题定义与总体结构

给定图像 $x \in \mathbb{R}^{3\times H\times W}$，按 $p\times p$ 切分为 $N = (H/p)(W/p)$ 个图像块（patch）。记编码器为 E、解码器为 R。本文的目标是训练一对 (E, R)，使 R 仅凭 E 输出的 K 个 token 重建整幅图像：

$$\begin{aligned} \hat{x} &= R\big(z_s\big), \\ z_s &= E_s(x) \in \mathbb{R}^{K\times D}, \qquad K < N, \end{aligned} \tag{1}$$

其中 D 为编码器隐层维度，z_s 称为**压缩表示**，K 称为 **token 预算**。重建质量用像素 L1 度量，压缩强度用压缩比 K/N 度量；在给定 K 下重建误差越低，说明 z_s 保留的图像信息越多。

图 1 给出总体结构。编码器只前向一次，把图像压缩为 K 个特殊 token；解码器把这 K 个 token

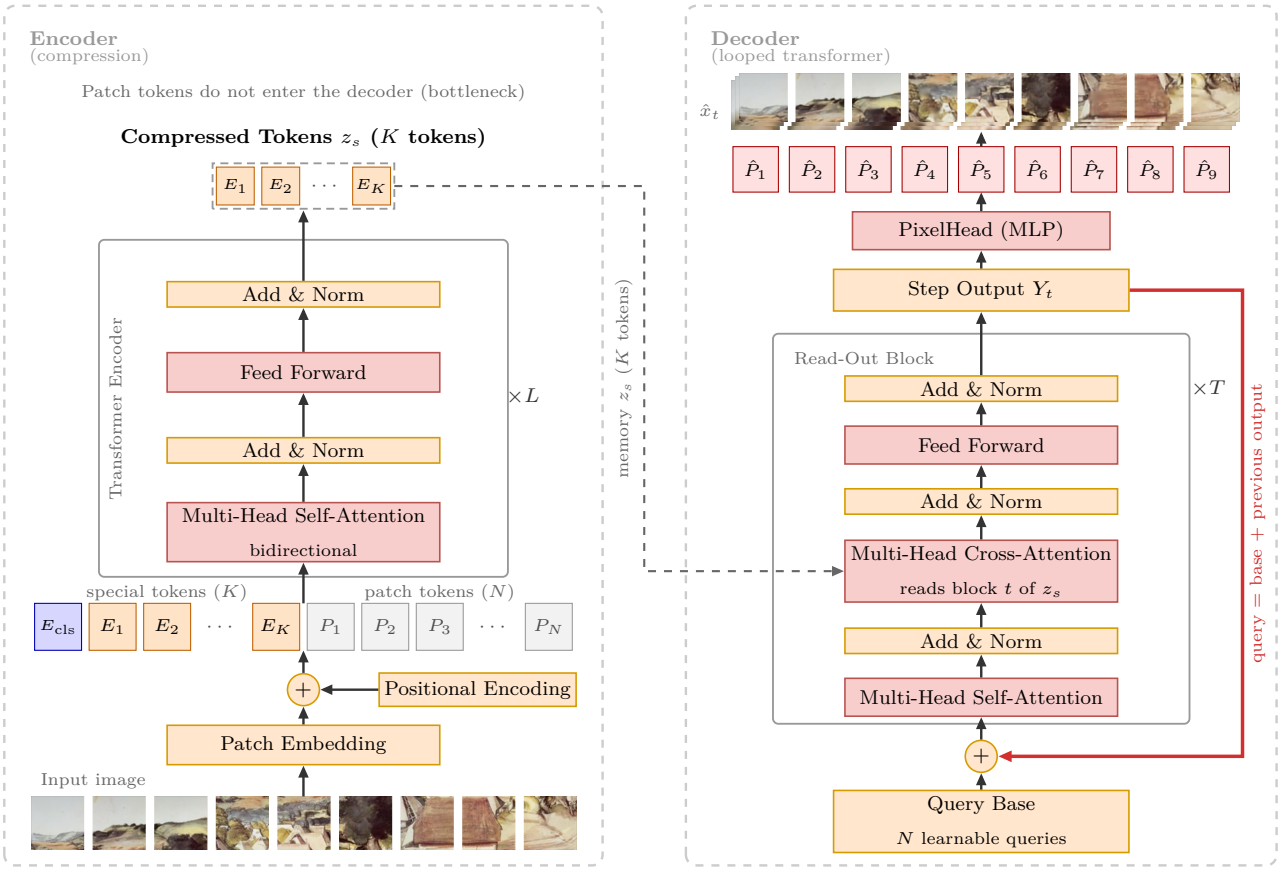


图 1　模型架构图

逐步读出并还原为像素。两部分通过同一个 z_s 相连。本文的全部训练信号来自像素重建，不引入分类、对比或语言监督。

在结构来源上，本文受自适应长度分词（ALIT）的循环分配[19]启发：两者都让同一组权重被反复施加，并让每一步可用的表示容量递增，因此同一个模型沿轨迹展开就得到一族长度不同的码。本文的取舍是将这一过程整体移至解码侧——编码器只前向一次，循环只发生在读出块 Φ 上——并以确定性的分块读窗口替代“每轮新增可学习 token”。这一取舍的动机与后果（尤其是“预算可自适应调整”与“容量可按图分配”之间的取舍）在第 5.3 节与 ALIT 逐项对照。

3.2　编码器：把特殊 token 放进输入序列

记分类 token 为 c，K 个特殊 token 为 $s_1, \ldots, s_K$，N 个 patch token 为 $e_1, \ldots, e_N$。编码器的输入是把三者直接拼接所得的序列：

$$X^{(0)} = \big[\, c;\ s_1 \cdots s_K;\ e_1 \cdots e_N \,\big], \qquad \big|X^{(0)}\big| = 1 + K + N, \tag{2}$$

其中 $e_n = \mathrm{PatchEmbed}(x)_n + \mathrm{PE}_n$ 由图像块嵌入与位置编码相加得到。编码器为 L 层双向 Transformer，层内自注意力不加任何掩码：

$$X^{(\ell)} = \mathrm{Layer}_\ell\big(X^{(\ell-1)}\big), \qquad \ell = 1, \ldots, L. \tag{3}$$

最后对末层输出做层归一化，并截取前 $1+K$ 个位置，得到解码器可用的压缩表示：

$$[\, z_{\mathrm{cls}};\ z_s \,] = \mathrm{LN}\big(X^{(L)}\big)_{0:1+K}, \tag{4}$$

其中 $z_{\mathrm{cls}} \in \mathbb{R}^{1\times D}$、$z_s \in \mathbb{R}^{K\times D}$。**patch token 不进入解码器**：它们只作为编码器内部的上下文，为特殊 token 提供可路由的内容；解码器只能看到式 (4) 给出的 z_s，瓶颈由此形成。特殊 token 与 patch token

在同一序列中做**全双向**注意力，每个特殊 token 既可以读取任意 patch，也可以与其他特殊 token 交换信息；因此不存在预先指定的 token 槽位，整幅图像在各特殊 token 上如何分摊，由 L 层交互共同决定。

3.3 特殊 token 的参数化

K 个特殊 token 共享同一个可学习内容向量，仅由各自的位置编码区分：

$$s_k = \theta_{\text{tok}} + \pi_k, \qquad k = 1, \ldots, K, \tag{5}$$

其中 $\theta_{\text{tok}} \in \mathbb{R}^D$ 为共享向量，π_k 为第 k 个位置的可学习位置编码。这一参数化使特殊 token 之间**不存在先验分工**：哪一个 token 承载图像的哪一部分信息，由式 (3) 的双向注意力在训练中决定。本文不另外引入路由机制，而是让解码器必须依赖编码器内部形成的路由，而不能依赖 token 与固定区域的对应关系。

3.4 解码器的记忆与查询基

解码器把编码器输出的 $K+1$ 个 token 作为记忆序列，并为其单独加一组位置编码 Π；同时持有一张与 patch 位置一一对应的可学习查询模板：

$$A = [\, z_{\text{cls}};\ z_s \,] + \Pi, \qquad Q_0 = q_{\text{base}} + q_{\text{pos}}, \tag{6}$$

其中 $A \in \mathbb{R}^{(K+1)\times D'}$、$Q_0 \in \mathbb{R}^{N\times D'}$，$D'$ 为解码器维度，$q_{\text{base}}, q_{\text{pos}} \in \mathbb{R}^{N\times D'}$ 为可学习参数，Q_0 的第 n 行对应第 n 个 patch 位置，行数恒为 N、与 K 无关。Q_0 是参数而非由输入导出的量，图像信息只能通过读取记忆 A 进入解码过程。

3.5 分块读窗口

第 i 步只读取记忆序列 A 的一个连续片段，称为**读窗口**。记读出步数为 T，第 i 步的窗口为 $[l_i, h_i]$，窗口序列取自位置 $0, \ldots, K$ 上的一个划分：

$$\begin{aligned} &l_1 = 0, \qquad l_{i+1} = h_i + 1, \qquad h_T = K, \\ &M_i = A[\, l_i : h_i \,], \qquad i = 1, \ldots, T. \end{aligned} \tag{7}$$

各窗口首尾相接、互不重叠，合起来覆盖全部 $K+1$ 个位置。T 与各窗口的宽度构成一个**读窗口计划**，它决定容量随步数增长的节奏。

本文主实验采用**均匀窗口**（fixed 计划）：读出步数固定为 $T = 16$，每步读 2 个特殊 token。第 1 步从位置 0 读起，因而同时看到 z_{cls} 与前 2 个特殊 token；第 $i \geq 2$ 步只读本步对应的分块，不再包含 z_{cls}。$K = 32$ 时 16 步恰好读完，第 t 步累计读到 $2t$ 个特殊 token，对应的预算为 $2, 4, \ldots, 32$。

这一划分替代了“每轮新增可学习 token”：容量随步数增长而不引入新参数，各步的读入量由窗口计划给出，不随图像内容调整（第 5.3 节）。均匀窗口使相邻档位之间的信息增量完全相同，便于按步截断；递增窗口则让前几步用更少的 token 先给出粗解。读出步数固定后，token 预算由 K 决定，而不是由步数决定。本文受控实验取 $K = 32$、$T = 16$。

3.6 循环读出更新

记解码块为 Φ，它是一个 L_{dec} 层的 Transformer 解码器堆叠，每层依次执行查询自注意力、对 M_i 的交叉注意力与前馈网络；Φ 的全部参数在所有读出步之间共享。第 i 步的查询 Q_i、输出 Y_i 由下式给出：

$$\begin{aligned} &Q_1 = Q_0, \qquad Q_i = q_{\text{base}} + Y_{i-1} \quad (i \geq 2), \\ &Y_i = \Phi\big(Q_i,\ M_i\big), \qquad i = 1, \ldots, T, \end{aligned} \tag{8}$$

其中 Q_0 与 q_{base} 由式 (6) 给出，M_i 由式 (7) 给出。式 (8) 中，每一步的输入查询由固定查询基与上一步的输出相加得到。该相加不经过投影或归一化，因此循环不引入额外参数，读出步数只增加计算量、不增加模型容量。

式 (8) 存在一个结构上的退化解：若解码块 Φ 趋近恒等映射，则 $Y_i \approx i\, q_{\text{base}} + q_{\text{pos}}$，循环只是在重复累加查询基，不携带新的图像信息。该退化解无需训练即可满足“输出幅度逐级变化”，因此逐步精炼只能由像素监督驱动，不能由结构本身保证。

3.7 逐步像素预测与深监督损失

每一步的表示输出经同一个两层 MLP 还原为该步对整幅图像的预测：

$$\hat{x}_i = \text{PixelHead}\big(\text{LN}(Y_i)\big) \in \mathbb{R}^{N\times 3p^2}, \tag{9}$$

其中 $i = 1, \ldots, T$，$3p^2$ 为一个 patch 的像素维数（$p = 16$ 时为 768），PixelHead 在所有步之间共享。**每一步都预测整幅图像**，而不是各步分工预测局部，因此“第 i 步的像素误差”是一个可直接观测的量。训练损失为各步像素 L1 的等权平均：

$$\mathcal{L}_{\text{rec}} = \frac{1}{T} \sum_{i=1}^{T} \text{MAE}\big(\hat{x}_i,\, x\big), \tag{10}$$

其中 $\text{MAE}(a, b)$ 为逐元素平均绝对误差（除以 $N \cdot 3p^2$）。式 (10) 对每一步等权、对每个像素等权，不使用步权重、区域掩码或额外正则项；本文的全部实验都使用这一形式。

3.8 推理时的自适应读出

由式 (8) 可知，Y_i 只依赖 $Y_1, \ldots, Y_{i-1}$，因此前 t 步的输出与完整 T 步前向中第 t 步的输出逐位相同。推理时把循环停在任意第 t 步，就得到一个使用 C_t 个 token 的压缩码，其中

$$C_t = h_t, \qquad t = 1, \ldots, T, \tag{11}$$

h_t 由式 (7) 给出：z_{cls} 占第 0 位、不计入 token 预算，故第 t 步读到的特殊 token 数为 h_t；本文主实验的 16 个预算依次为 $2, 4, \ldots, 32$（每步新增 2 个）。于是同一个训练好的模型无需重训即可在多个 token 预算下工作。提前停止只改变推理时执行的步数，不改变前向计算图；每步都要完整执行一次 Φ，因此计算量随预算线性增长而非次线性增长，这也是读出步数可变的代价（第 5 节）。

3.9 训练流程

训练分两个阶段。第一阶段只训练单步解码：令 $T = 1$、读窗口取 $[0, K]$，即解码器一次性读完全部特殊 token，得到一个标准的输出查询解码器。第二阶段从第一阶段的权重热启动，切换为 T 步循环，并按式 (10) 施加逐步监督。热启动的理由是：多步循环中前几步只读到很少的特殊 token（2, 4, 6, …），此时条件最优解接近图像均值，从零训练多步容易停留在该平凡解；先用单步全读把 (E, R) 训练到非平凡初值，再切换到多步，可显著减少脱离该平凡解所需的步数（第 4.1 节）。

循环状态的梯度**沿时间反向传播**（back-propagation through time, BPTT）：式 (8) 中的相加不施加停止梯度（stop-gradient），因此第 i 步的误差可以回流到前面各步的输出与查询基。本文所有实验都使用全步反传。

4 实验

本节在 ImageNet-100 上做受控实验。基准配置取 240M 档：编码器与解码器都是标准 Transformer（softmax 注意力），可训参数分别为 244.43 M 与 237.11 M，体量基本对齐，因此不同设置之间的差异可以归因到结构因素；除特别说明外，所有比较都在同一训练协议内进行。这一档位不是预先指定的，而是由第 4.4 节的容量阶梯选出——在本协议下容量收益在 240M 档达到峰值，继续加宽反而回落。

4.1 实验设置

数据与模型. 数据为 ImageNet-100，训练集 121,689 张、验证集 5,000 张；输入 224×224，patch 为 16×16，故 $N = 196$；token 预算 $K = 32$，以 token 数计的压缩比 $N/K = 6.12$。编码器为 13 层 Transformer，隐层维度 1248、16 头，可训参数 244.43 M；解码器为 7 层 Transformer 解码器堆叠，隐层维度 1440、16 头，可训参数 237.11 M；两者合计 481.54 M，体量相当（244.43 M 对 237.11 M），因此两者之间的比较不涉及“模型更大”的解释。每个特殊 token 是 1248 维连续向量，32 个 token 共 39,936 个连续值。主要配置见表 1。

训练与指标. 训练分两阶段。阶段 1 为单步解码：$T = 1$、读窗口取 $[0, K]$（一次读完全部 32 个特殊 token），45 个 epoch、batch size 128、共 42,750 步，从零训练；阶段 2 为 16 步循环：从阶段 1 的最优权重热启动，采用均匀窗口（每步 2 个特殊 token），6 个 epoch、batch size 64、共 11,406 步，编码器与解码器联合微调。优化器为 AdamW，学习率 2×10^{-4}，cosine 调度并带 8% warmup，权重衰减 0.05，梯度裁剪 1.0，全程 bf16。评价指标

以峰值信噪比（PSNR）为主：在 $[0,1]$ 空间中逐图计算、再对 5,000 张留出图取平均，单位 dB，越高越好；辅以结构相似度（SSIM）与像素 L1（0–255 空间，不裁剪）。三者都是逐图配对指标，直接比较每张重建图与它对应的 GT，衡量的是信息被保留下来的程度。本文不以重建 FID（rFID）为主指标：它以 InceptionV3 特征在整批重建图与整批 GT 图之间比较均值与协方差，是 unpaired 的集合级分布度量，不含逐图重建精度（一批“逐图全错、但都取自自然图像分布”的输出也能得到很低的 rFID），且对纹理统计的权重远大于几何误差；它适合评价生成式模型的真实感，不适合本文的信息保持问题。

瓶颈臂. 为检验每 token 维数的影响，本文在 240M 档上另训两支瓶颈臂：在编码器末端加一层线性映射，把 1248 维的特殊 token 分别压到 448 维与 224 维（z_s 的存储值数由 39,936 降到 14,336 与 7,168，即 2.79 倍与 5.57 倍）；两支臂均从 16 步模型热启动、全量微调 3 个 epoch（batch size 64、学习率 2×10^{-4}、cosine 调度，其余同阶段 2），可训参数分别为 481.51 M 与 480.63 M，与主模型（481.54 M）基本一致。每步仍读 2 个特殊 token，故第 t 步累计 $896t$ 与 $448t$ 个连续值，16 步的上限分别为 14,336 与 7,168 个值。

参考基线. 本文不使用量化与熵编码，“压缩率”以 token 数而非比特数度量。不随模型结构变化的两条像素基线为：逐 patch 直流（每个 patch 用其均值表示，18.45 dB）与常量灰（11.81 dB），用于判断模型是否真正学到了图像信息。第 4.2 节另用逐块 PCA 作为有学习基的线性对照：基在训练集 patch（400 张图的 78,400 个 patch）上学习、全图共享、不逐图拟合，每块保留前 r 个主成分，故每图信息量为 $196r$ 个连续值。

口径说明. 本文主实验的 16 个预算依次为 $2,4,\ldots,32$ 个特殊 token（第 t 步累计 $2t$ 个）。对照统一按**每图连续值个数**对齐：主模型每 token 1248 维，32 个 token 共 39,936 个值；448/224 维瓶颈臂每图最多 14,336 与 7,168 个值（第 t 步累计 $896t$ 与 $448t$）；逐块 PCA 每图 $196r$ 个值，取 $r = 9/18/37/73$ 与瓶颈臂对齐（1,764/3,528/7,252/14,308，相对差 $\le 1.6\%$）。图 3、图 4 取自主模型（$K = 32$、$T = 16$），图 2 为 224 维瓶颈臂。压缩率以 token 数度量：旧 63M 基准为 3.06×（196→64），本文 240M 档为 6.12×（196→32）；以连续值个数计时，主模型每图 39,936 个值、瓶颈臂为 14,336/7,168 个值——两种口径在文中分别标明，不混用。

表 1 实验配置

项目	设置
数据	ImageNet-100（121,689 / 5,000）
输入 / patch	224×224 / 16×16，$N = 196$
token 预算	$K = 32$，压缩比 $N/K = 6.12$
每图信息量	$32\times1248 = 39{,}936$ 个连续值
编码器	13 层，$D = 1248$，16 头，244.43 M
解码器	7 层，$D' = 1440$，16 头，237.11 M
总参数	481.54 M
阶段 1	单步（$T = 1$），45 epoch，bs 128，42,750 步
阶段 2	16 步（每步 2 token），6 epoch，bs 64，11,406 步
优化器	AdamW，lr 2×10^{-4}，warmup 8%
精度 / 硬件	bf16 / A800 80 GB
评价指标	PSNR（主）、SSIM、L1（0–255）

4.2 与线性码的预算分层对比

本节按“每图连续值个数”把本文的码与逐块 PCA 基线对齐：同一份 5,000 张留出图、同一评测口径（逐图 PSNR 为主，SSIM 与像素 L1 为辅），数值见表 2 与图 2。为覆盖不同预算区间，这里同时使用三支模型：主模型（每 token 1248 维，最多 39,936 个值）与两支瓶颈臂（每 token 448/224 维，最多 14,336/7,168 个值，见第 4.1 节）。其他分词器不在本表并列：TiTok 与 ALIT 都带 VQGAN 量化底座，ALIT 还含熵编码与对抗损失，训练数据、重建分辨率与留出集也各不相同，数值直接并列不可靠；本文不作跨方法的数值对比，待按统一协议复现其训练与评测后再行比较。

由表 2 与图 2 可得四点。第一，**紧预算端本文的码更好**：在 1,792–3,584 个连续值处，448 维臂为 23.19 / 24.99 dB、224 维臂为 23.66 / 25.21 dB，而

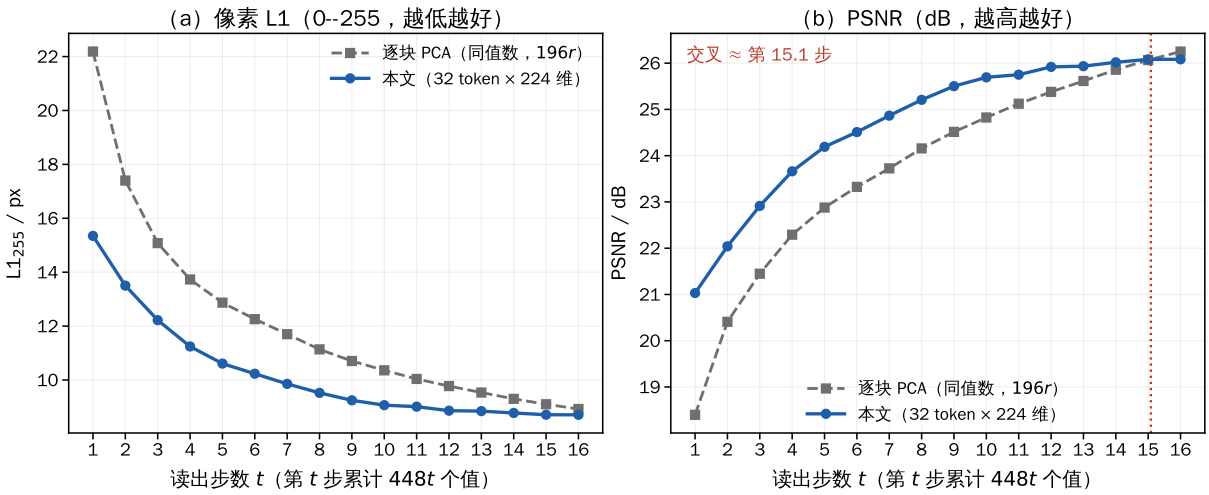


图 2　按读出步数比较像素 L1（左）与 PSNR（右）：本文 224 维臂与同值数逐块 PCA。横轴为读出步数，每步读 2 个 224 维 token（448 个值），第 t 步累计 $448t$ 个值；PCA 按 $r = 448t/196$ 在实测的 rank 1–40 之间线性插值，与本文严格同值数。实线为本文，虚线为逐块 PCA；L1 上本文 16 步全程更低，PSNR 上第 1–15 步本文更高、约在第 15.1 步交叉。

表 2　预算分层的同值数 PSNR 对照（dB）

方法	每图连续值个数			
	1,792	3,584	7,168	14,336
本文 448 维臂（4/8/16/32 token）	23.19	24.99	26.63	27.69
本文 224 维臂（8/16/32 token）	23.66	25.21	26.08	/
逐块 PCA（rank 9/18/37/73）	22.26	24.10	26.28	29.06

注：5,000 张留出图上逐图 PSNR 的均值，越高越好。两支瓶颈臂在编码器末端各加一层线性映射，把 1248 维的特殊 token 压到 448 / 224 维；每步仍读 2 个 token，故第 t 步累计 $896t$ / $448t$ 个值。列头四档分别对应 448 臂的 $t = 2, 4, 8, 16$ 与 224 臂的 $t = 4, 8, 16$（224 臂的上限为 7,168 值）。逐块 PCA 每图为 $196r$ 个值，$r = 9/18/37/73$ 对齐四档（1,764/3,528/7,252/14,308，相对差 $\leq 1.6\%$），共享基（768 × 256）算模型参数、不计入每图预算。相同值数下两臂读入的 token 数不同（7,168 值处 448 臂 16 个、224 臂 32 个），比较时须注意这一结构差异。

同值数 PCA（rank 9/18）为 22.26 / 24.10 dB，即本文高 +0.89 ∼ +1.41 dB；L1 同向（11.78 / 11.24 对 13.79 px，9.67 / 9.52 对 11.21 px）。原因是预算极紧时逐块 PCA 的每个 patch 只保留 9–18 个主成分、几乎没有容量，而本文的码把整幅图像的信息分摊到 32 个 token 上，并由像素监督学到跨位置的先验，因此在极紧预算下更省。

第二，**预算放宽后线性码反超**：7,168 值处 448 维臂仍高 0.34 dB（26.63 对 26.28 dB），224 维臂的 PSNR 为 26.08 dB、低 0.20 dB，但其 L1 低 0.18 px（8.71 对 8.89 px）；到 14,336 值处 PCA 反超 1.37 dB（29.06 对 27.69 dB）。主模型自身在同值数下也不占优：在 4,992–39,936 值的四档上 PCA 均更高，39,936 值处为 27.94 dB 对 35.75 dB。这与“逐块 PCA 保留 196 个空间位置的码、只降低每个位置的维数”一致：预算越松，它越接近逐位置线性编码的优势区间。

第三，图 2 把 224 维臂与同值数 PCA 按读出步数逐点比较：每步读 2 个 224 维 token（448 个值），第 t 步累计 $448t$ 个值；PCA 在实测的 rank 1–40 之间按 $r = 448t/196$ 线性插值，与本文严格同值数。L1 上本文 16 步全程更低，优势由第 1 步的 −6.84 px 单调收窄到第 16 步（7,168 值）的 −0.22 px，始终没有交叉；PSNR 上第 1–15 步本文更高（+2.63 降到 +0.02 dB），第 16 步 PCA 反超 0.17 dB，交叉点约在第 15.1 步。两个指标的交叉位置不同，是因为 PSNR 以均方误差为底、对少数大误差像素更敏感，而 L1 是逐像素平均；因此“紧预算端本文更好”的成立范围与指标有关：按 L1 可写到 7,168 值，按 PSNR 只到约 6,720 值（第 15 步）。

第四，**压窄每 token 的维数代价很小**：把每 to-

ken 由 1248 维压到 448 维后，每图连续值由 39,936 降到 14,336（36%），末步 PSNR 仅由 27.94 降到 27.69 dB；在同一 14,336 值下比旧 63M 基准（24.15 dB）高 3.54 dB。这说明 1248 维中的冗余较大、读出质量的主要瓶颈在 token 数而非每 token 的宽度。相同值数下“少而宽”与“多而窄”的取舍并不对称：7,168 值处 448 维臂（16 个 token）为 26.63 dB，高于 224 维臂（32 个 token）的 26.08 dB，说明在本协议里把预算摊到更多 token 上并不必然更好。

因此本文的定位不是“同值数下最优的像素重建器”，而是“单一模型中可截断的 32-token 语义码”：在不增加参数、不重新训练的前提下，一个 32-token 的表示覆盖由少到多的多档预算，而逐块 PCA 既没有减少 token 数，也不能按步截断到更少预算；在极紧预算端（$\leq$ 3,584 值）本文的码更好，在较松预算端线性码更好。本文暂不与其他分词器并列数值：TiTok、ALIT 等带量化码本、熵编码与对抗损失，训练数据、分辨率与评测集都不同，直接比较不可靠；待按统一协议复现其训练与评测后再行比较。

4.3 图像复杂度

读出步数增加会改善重建，但不同图像是否同等受益、容量又如何影响这一差异，仍需检验。本小节使用公开的 SAVOIAS 人评复杂度数据[37]作为分布外分析集：People-Art 420 张、Places 200 张、MSCOCO 200 张，复杂度分数为 0–100 的众包成对比较结果。被测模型为 ImageNet-100 上训练的 240M 档（$K = 32$、$T = 16$），未在 SAVOIAS 上微调。为把“容量”与“预算”两个因素分开，本小节同时用到第 4.4 节的 5 个容量档（合计 63.0 M、119.3 M、238.2 M、481.5 M、720.6 M），它们与基准同协议（$K = 32$、$T = 16$、每步 2 个 token），唯一自变量是宽度。

图 3 给出 240M 档的热图：列为 People-Art、Places、MSCOCO，行为 token 预算（2 到 32，第 t 步累计 $2t$ 个特殊 token），横轴为人评复杂度（15 到 85），颜色为像素 L1（0–255，逐图高斯平滑，$\sigma = 5$），越深误差越大。两个方向与旧协议（63M/K=64）一致：固定 token 预算时，复杂度越高误差越大；固定复杂度时，token 越多误差越低，且高复杂度一列下降得更快。

三个子集上复杂度与重建误差在全部 16 个预算上正相关：最少预算（2 token）时 $\rho = +0.520$、$+0.576$、$+0.415$（People-Art、Places、MSCOCO），最多预算（32 token）时 $\rho = +0.518$、$+0.477$、$+0.327$；ρ 随预算只轻微下降，说明复杂度效应不是某一档预算的偶然。把容量维度也纳入（第 4.4 节的 5 个容量档 $\times$ 3 个子集 = 15 个组合），15 格全部为正（$\rho \in [+0.327, +0.536]$），并随容量只有轻微下降（子集平均从 63M 档的 0.489 降到 482M 档的 0.441，721M 档回升到 0.453）。

按复杂度三分位分组，240M 档上最高组与最低组的末步 L1 差为 3.629 px（People-Art）、2.821 px（Places）、1.504 px（MSCOCO）。容量增加缩小的是这一差的绝对幅度：从 63M 档的 5.655 / 4.992 / 2.811 px 降到 482M 档的 3.629 / 2.821 / 1.504 px（降幅 36% / 43% / 47%）；但相对幅度基本不变（482M 档分别为 66.7%、49.7%、22.5%）。也就是说，容量买到的是整体误差水平的下降，“越复杂越难”的相对结构没有被容量抹平。

在分布外数据上，240M 档末步的 L1 为 7.231、PSNR 为 27.406 dB、SSIM 为 0.8257，与同协议 ImageNet-100 验证集上的 7.004、27.936 dB、0.8288 接近（L1 差 +0.227 px、PSNR 差 −0.53 dB）；OOD 上的最优停步为第 15 步，与 ID 一致（5 个容量档依次为 11 / 13 / 13 / 15 / 16 步）。逐步曲线在第 15 步之后基本饱和，说明该收敛位置由读出协议（每步 2 个 token、$T = 16$）而非数据分布决定。

图 4 给出低、中、高复杂度图像在 240M 档上的重建示例：三行的复杂度依次为 10、48、89（文件 138/15/21.jpg），列为原始图像与第 1、2、4、8、16 步的重建（累计 2、4、8、16、32 个 token）。复杂度 10 的图像在第 1 步（2 token）已还原主要物体与布局（L1 5.27 px），继续读入主要改善边缘与

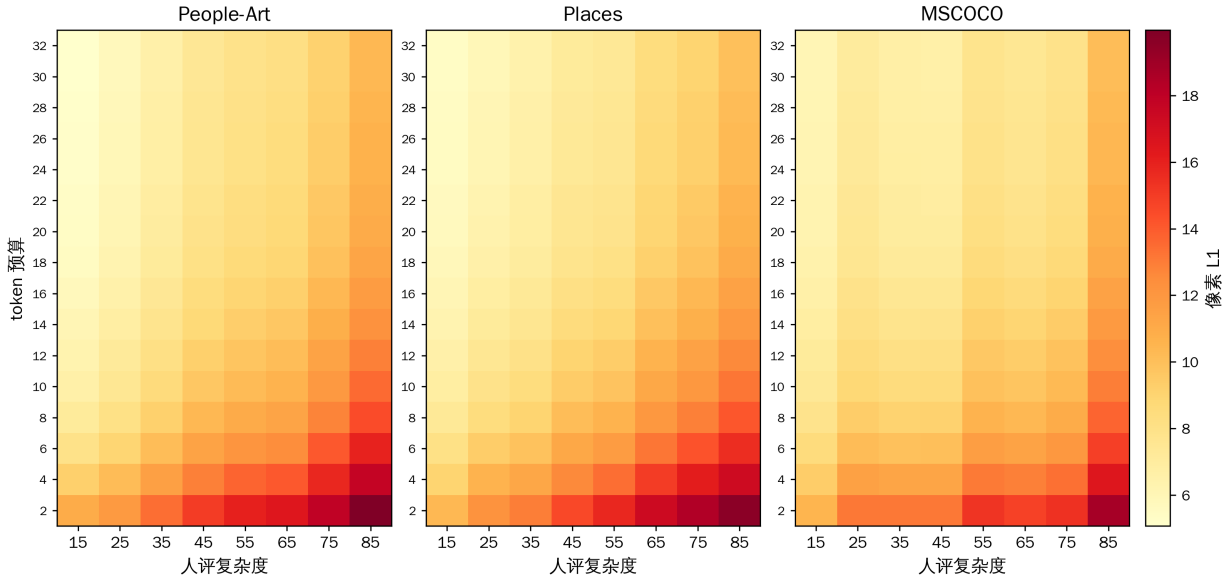


图 3 复杂度热图（240M 档，16 档预算）

纹理（末步 2.38 px）；复杂度 48 与 89 的图像在第 1 步只有色块与大致明暗（14.53 / 14.81 px），到第 4 步（8 token）区域边界与物体轮廓才清晰（9.76 / 10.69 px），末步降到 6.50 / 7.60 px。低复杂度行的相对降幅与另两行相近（55% 对 55%、49%），但绝对改善明显更小（2.89 对 8.03、7.21 px）：复杂度主要抬高误差的起点与整体水平，读出步数的收敛节奏没有被改变。

4.4 容量阶梯

本文的基准配置由容量阶梯选出。前面的实验固定 $K = 32$、$T = 16$、每步 2 个特殊 token，本小节只改变编码器与解码器的宽度，得到 5 个容量档：编码器与解码器各约 30 M（合计 63.0 M）、60 M（119.3 M）、120 M（238.2 M）、240 M（481.5 M）与 360 M（720.6 M）。深度固定（编码器 13 层、解码器 7 层），只加宽隐层；5 档共用同一训练日程（阶段 1 单步 45 epoch、阶段 2 16 步 6 epoch），容量是唯一自变量。结果见图 5。

$T = 16$ 末步的 PSNR 依次为 24.148、25.747、27.308、27.936、27.314 dB，归一化空间的 L1 依次为 0.1877、0.1570、0.1320、0.1215、0.1305，SSIM 依次为 0.6638、0.7389、0.8039、0.8288、0.8015；相邻档的边际收益为 +1.60、+1.56、+0.63、−0.62 dB，按每 100 M 参数折算依次为 +2.84、+1.31、+0.26、−0.26 dB/100M。由此得到三点。第一，**容量甜点在 240M 档（合计 481.54 M）**：从 63M 到 240M 用 418 M 参数买来 +3.79 dB，而从 240M 到 360M 用 239 M 参数反而亏 0.62 dB；L1 与 SSIM 同向，说明这不是单一指标的噪声。第二，**阶段 1 与阶段 2 同序**：两阶段的末步 PSNR 分别为 28.728 / 30.242 / 31.205 / 32.345 / 31.977 dB 与 24.148 / 25.747 / 27.308 / 27.936 / 27.314 dB，B 相相对 A 相的落差在 4.10–4.66 dB 之间几乎不随容量变化，说明“每步 2 个 token、16 步”的窄窗口读出损失不是容量问题，加参数填不平。第三，**容量抬高的是每一步的整体水平，而非收敛速度**：第 1 步的 L1 从 17.18 px 降到 13.50 px，而最优停步只在 11 / 13 / 13 / 15 / 16 之间后移。

作为旁证，在更早的 63M/K=64、8 步递增窗口协议下还做过 token 预算的扫描（$K = 8$ 到 128，共 9 档，每档单独训练）：$K \le 64$ 时质量单调上升（PSNR 由 21.78 dB 升到 27.76 dB），$K > 64$ 后回落（$K = 96$ 为 26.24 dB、$K = 128$ 为 25.09 dB），因此旧协议的预算操作点是 $K = 64$。该结果

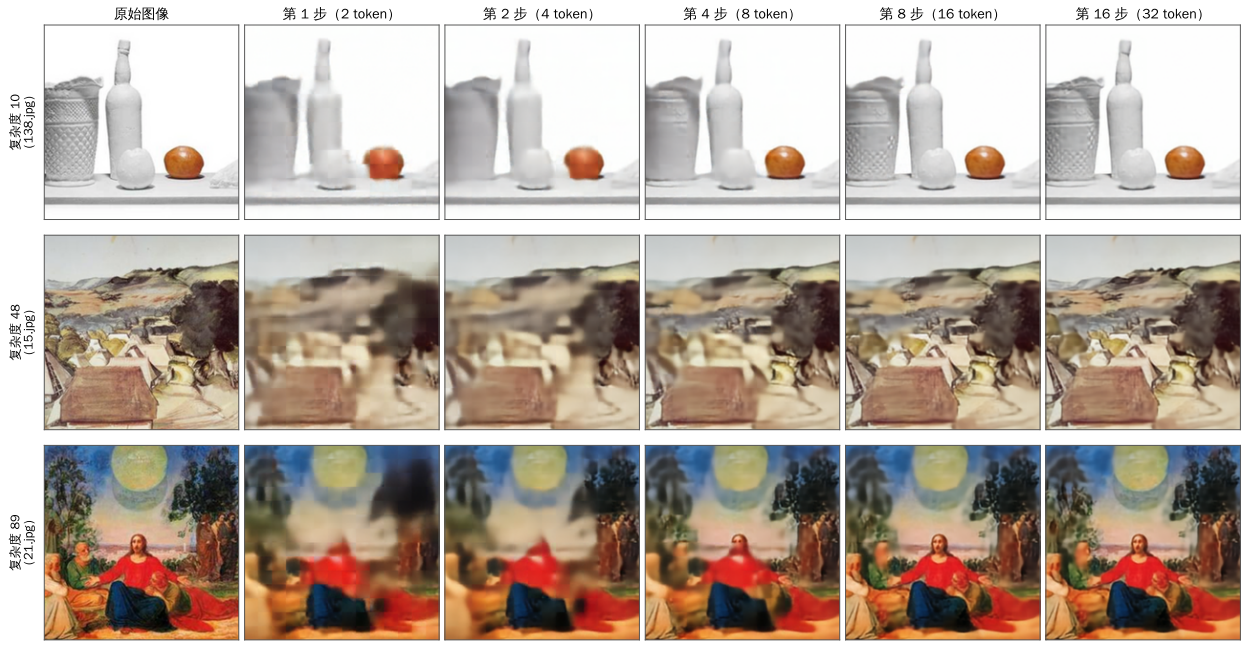


图 4　重建示例（240M 档，$K = 32$、$T = 16$；列为原始图像与第 1、2、4、8、16 步（2–32 token）的重建）

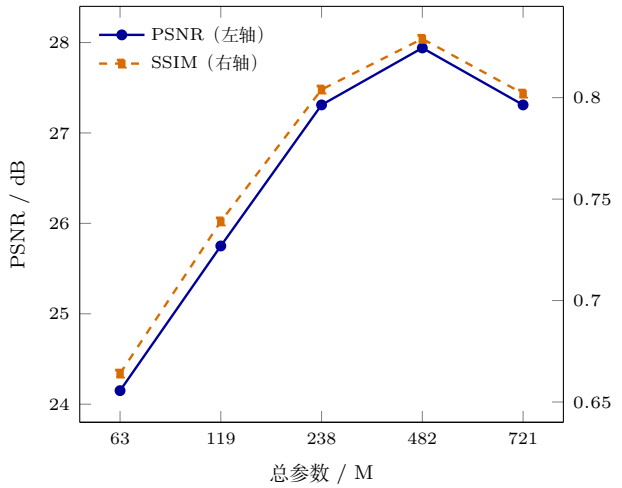


图 5　容量阶梯

与新基准不可直接相减（模型宽度、步数与窗口计划都不同），只作为“预算需与容量、步数匹配”的历史旁证保留；新基准下预算与步数的匹配关系是 $K = 2T$。

5　讨论

5.1　循环位置与容量增长

第 4.2 节的对照可以归结为一个位置选择。ALIT 的循环在编码侧：每轮都把图像与既有的隐 token 一并重新编码，并按上一轮的重建质量掩掉已蒸馏充分的部分；本文的循环在解码侧：编码器只前向一次，每步只运行读出块 Φ。这一取舍把编码器的开销与读出步数解耦，使任意预训练编码器都可以直接接入，而不必为每个 token 预算重新编码；代价是编码器不具备“因局部区域重建不足而重新组织输入”的通路。

位置选择还决定了容量如何增长。ALIT 每轮拼接新的可学习 token，新 token 构成新的记忆槽，可在后续轮次中专门承载某一区域，这是它报告 token 专业化与目标发现的来源。本文的容量增长是确定性的分块窗口：token 集合固定为 K，第 t 步仅增加可读入的 token 数。由此得到两点性质：不引入额外参数；第 t 步的码与完整前向中第 t 步逐位相同（式 (8)），因此停在任意一步的结果是精确的。代价是第 4.3 节观察到的复杂度效应无法转化为逐图的预算差异——所有图像共用同一条逐步曲

线，仅高度不同。

5.2 预算的取用与匹配

第 4.3 节表明，复杂图像在每一项容量档上的误差都更高，容量增加缩小的是复杂度惩罚的绝对幅度：最高与最低三分位的 L1 差从 63M 档的 5.655 / 4.992 / 2.811 px 降到 240M 档的 3.629 / 2.821 / 1.504 px，但相对差基本不变（240M 档仍为 66.7% / 49.7% / 22.5%）。这说明“不同图像对预算的需求确实不同”的动机成立，但相对结构不随容量改变；同一条逐步曲线的形状对所有图像相同，逐图差异只体现在高度上。

因此本文的建议是：**由部署方按场景精度预设预算**，而不是让模型在推理时逐图预测预算。例如低精度场景取 2–8 个 token，高精度场景取 16–32 个 token。这一建议与“预算可自适应调整”的设计一致：用户只需训练一个模型，就能在多个预设预算下取用。若要进一步按图分配，需要额外的判断通路，本文未做。

第 4.4 节还给出两条可操作的边界：一是容量，在本协议下 240M 档（合计 481.5 M）是甜点，继续加宽到 721M 反而退化；二是步数与预算的匹配，主实验取 $K = 2T$（每步 2 个 token），使每一步都有实质的信息增量。

5.3 同类机制的异同

第 4.2 节只把逐块 PCA 并入按预算分层的同值数对照（其他分词器的口径差异见该节说明），本文与 ALIT 的机制差别集中在三处。

循环放在哪一侧。 ALIT 的循环在编码侧，本文在解码侧，取舍见第 5.1 节。

容量如何增长。 ALIT 每轮新增记忆槽，本文用读窗口扩张；前者可让 token 分工，后者不增参数，且停在任意一步的结果精确。

码是否量化。 ALIT 量化到共享的一维码本，“长度”同时是比特预算，可与 VQGAN、TiTok 在 FID 与 bpp 上比较；本文不做量化、不建熵模型，压缩率只以 token 数度量，因此第 4.2 节的 PSNR 对照只在本协议内成立，不能换算成比特率；与其他分词器的数值比较需在统一协议下复现其训练与评测后再做。

这三点共同说明：两者都在“权重共享的循环 + 每步容量递增”这一框架下工作，但一个把容量放在写侧，一个放在读侧，因而得到的性质不同——前者可以按图分配长度，后者把预算的决定放在推理期。

5.4 适用边界

本文的结果有若干边界。第一，受控模型为 481.54 M 参数（编码器与解码器各约 240 M）、在 ImageNet-100 上从零训练，其绝对重建质量不能外推到大规模预训练模型；本文只在同一协议内比较，不对绝对数值作跨协议推广。第二，本文不做量化与熵编码，“压缩率”以 token 数度量，不是真实比特率；本文的模型底座（无 VQGAN、无对抗损失）、训练数据与分辨率也与 TiTok、ALIT 不同，因此表 2 不与其并列数值，待按统一协议复现后再比。第三，评价指标以像素重建为主，没有下游任务；压缩表示对图像描述、视觉问答等任务是否够用仍需另行检验。第四，单步与循环的训练预算未对齐（阶段 1 为 45 epoch 从零，阶段 2 为 6 epoch 热启动），本文不主张“循环优于单步”；同预算对照（从同一单步权重分别跑“继续单步”与“BPTT”）是下一步工作。第五，复杂度分析使用分布外的 SAVOIAS 数据，结论只说明“在该人评复杂度轴上呈对齐趋势”，不能推断 ImageNet-100 验证集内部的复杂度分布。第六，第 4.4 节的容量阶梯在 360M 档出现反转，原因（同一学习率与同一日程下宽模型更难优化）尚未与数据侧因素分离；旧协议（63M/K=64）下 $K > 64$ 的退化同理未完全解耦。第七，本文只检验了 ALIT 所列四条轴中的一条（复杂度），没有检验“对训练分布的熟悉度”，也没有分析“token 专业化”；对这两点本文既不主张也不否定。第八，按预算分层的对照（表 2）表明：预算越紧本文的码越占优，预算越松逐块线性码越占优；本文的主预算点（39,936 值）并非同值数最优（27.94 dB 对 PCA 的 35.75 dB），

因此本文不把“同值数下最优”作为主张。

5.5 时序的延伸

读出本来就是一条时间轴：第 t 步在第 $t-1$ 步的状态上再加一步。单帧里它对应“同样的内容看得更全”，放到视频上则对应“时间由过去到现在”。这是本文最看好的一条延伸：在视频里，循环不只是把一步拆成几步，而是让前一帧的结果成为后一帧的起点。

视频的相邻两帧几乎总是相似的：机位不动时背景逐像素不变，只有物体和光照在动；手持拍摄时差异也只集中在画面的一小部分。后一帧的大部分内容前一帧已经解释过，从零重建等于把同一块背景的 token 预算再花一遍。

一种自然的用法是：每帧只走少量步数，第一步的初值取上一帧的读出状态，由它承担“没变的部分”，新增的几步只补“变了的部分”。帧间差异小就少走几步，运动大就多走几步，镜头切换时退回全量读出。第 5.1 节的两点性质在这里正好合用：提前停止是精确的，少走几步不会改动已读出的结果；循环在解码侧，跨帧搬过去的只是一个读出状态，不必把上一帧重新编码一遍。

这种复用与视频编码器的帧间预测不在同一层：后者在比特层压掉静止背景，管的是码率；前者在特征层复用状态，省的是 token 预算。

真要走到这一步，还差三件事。一是训练：监督只在单帧内由粗到细，帧间状态没进过训练目标，需要把相邻帧接成序列、跨帧传状态后再微调。二是误差累积：上一帧的偏差会带进下一帧，系统要周期性插关键帧刷新。三是评价：视频还要看时序一致性，本文没有这项指标与视频数据。

6 结论

本文提出循环读出压缩：编码器在输入序列中携带 K 个可学习特殊 token，以像素重建为训练信号把整幅图像压缩到这些 token 上；解码器以权重共享的 Transformer 块反复读出，第 t 步只读取其读窗口，查询由固定查询基与上一步输出相加得到，每一步都直接预测整幅图像。该结构使 token 预算与读出步数绑定，同一模型可在推理时停在任意一步。结构上它受自适应长度分词（ALIT）的循环分配[19]启发：同样以“权重共享的循环 + 每步容量递增”换取一族长度不同的码，但循环被移到解码侧，容量增长由确定性的分块窗口给出（第 5.3 节）。

在 ImageNet-100 上的受控实验给出三点结论。第一，在 240M 档（编码器与解码器各约 240 M，合计 481.54 M）上，以 token 数计的压缩率由旧 63M 基准的 3.06 倍（196→64）提高到 6.12 倍（196→32），单步读出达到 32.35 dB，显著高于逐 patch 直流基线（18.45 dB），说明 32 个特殊 token 可以保留整幅图像的主要信息；把同一模型展开到 16 步，PSNR 由 22.07 dB 升到 27.94 dB。与逐块 PCA 的同值数对照按预算分层：在 $\leq$ 3,584 个连续值的紧预算端，本文 448 维与 224 维瓶颈码均高于同值数 PCA（$+0.89 \sim +1.41$ dB）；7,168 值处 448 维码仍高 0.34 dB（224 维码仅 L1 更低）；预算放宽到 14,336 值及以上时 PCA 反超（+1.37 dB；主模型在 39,936 值处为 27.94 dB 对 35.75 dB）——逐块 PCA 保留 196 个空间位置，预算越松越占优。把每个特殊 token 由 1248 维压到 448 维（每图连续值降为 36%）只损失 0.25 dB，且在同一 14,336 值下比旧 63M 基准高 3.54 dB，说明这类码的冗余主要在每个 token 的宽度方向。第二，人评复杂度与重建误差在三个子集上均正相关（$\rho = +0.33 \sim +0.54$，5 个容量档与 3 个子集共 15 个组合全部为正），容量缩小的是复杂度惩罚的绝对幅度而非相对结构；SAVOIAS 上的分布外结果与 ImageNet-100 验证集接近，最优停步同为第 15 步。第三，容量阶梯在 240M 档达到峰值（27.94 dB），继续加宽到 720.6 M 反而回落到 27.31 dB，240M 是本协议下的操作点；每步 2 个 token、16 步的读出损失（相对单步读满 32 个 token）在 4.10–4.66 dB 之间，且几乎不随容量变化。

本文的结论限定在“重建”这一信息保持口径内。与现有分词器在同一协议下的逐档对比、把该压

缩表示接到下游视觉—语言任务、把读出状态跨帧传递到视频流、引入量化与熵编码以给出真实比特率、以及在同预算下比较单步与循环，是后续工作；在参数量对齐下比较“每轮新增记忆槽”与“每步扩大读窗口”两种增容方式，则是判定本文与 ALIT 差异来源的直接实验。

参 考 文 献

[1] DOSOVITSKIY A, BEYER L, KOLESNIKOV A, et al. An image is worth 16×16 words: Transformers for image recognition at scale[C]//Proceedings of International Conference on Learning Representations. 2021.

[2] RADFORD A, KIM J W, HALLACY C, et al. Learning transferable visual models from natural language supervision[C]//Proceedings of International Conference on Machine Learning. 2021: 8748-8763.

[3] LI J, LI D, SAVARESE S, et al. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models[C]//Proceedings of International Conference on Machine Learning. 2023: 19730-19742.

[4] ALAYRAC J B, DONAHUE J, LUC P, et al. Flamingo: A visual language model for few-shot learning[C]//Advances in Neural Information Processing Systems: Vol. 35. 2022: 23716-23736.

[5] LIU H, LI C, WU Q, et al. Visual instruction tuning[C]//Advances in Neural Information Processing Systems: Vol. 36. 2023.

[6] LI Y, WANG C, JIA J. LLaMA-VID: An image is worth 2 tokens in large language models[A]. 2023. arXiv: 2311.17043.

[7] ZHANG S, FANG Q, YANG Z, et al. LLaVA-Mini: Efficient image and video large multimodal models with one vision token[C]//Proceedings of International Conference on Learning Representations. 2025.

[8] JAEGLE A, BORGEAUD S, ALAYRAC J B, et al. Perceiver IO: A general architecture for structured inputs and outputs[C]//Proceedings of International Conference on Learning Representations. 2022.

[9] YU Q, WEBER M, DENG X, et al. An image is worth 32 tokens for reconstruction and generation [C]//Advances in Neural Information Processing Systems: Vol. 37. 2024.

[10] YU L, LEZAMA J, GUNDAVARAPU N B, et al. Language model beats diffusion—tokenizer is key to visual generation[C]//Proceedings of International Conference on Learning Representations. 2024.

[11] DARCET T, OQUAB M, MAIRAL J, et al. Vision transformers need registers[C]//Proceedings of International Conference on Learning Representations. 2024.

[12] FAN X, et al. FALCON: Learning visual registers for efficient multimodal large language models[C]// Proceedings of the IEEE/CVF International Conference on Computer Vision. 2025.

[13] GE T, JING H, WANG J, et al. In-context autoencoder for context compression in a large language model[C]//Proceedings of International Conference on Learning Representations. 2024.

[14] YE X, GAN Y, HUANG X, et al. VoCo-LLaMA: Towards vision compression with large language models [A]. 2024. arXiv: 2406.12275.

[15] CAI M, YANG J, GAO J, et al. Matryoshka multimodal models[A]. 2024. arXiv: 2405.17430.

[16] GE Y, ZHAO S, ZENG Z, et al. Making LLaMA see and draw with SEED tokenizer[A]. 2023. arXiv: 2310.01218.

[17] WU Y, ZHANG Z, CHEN J, et al. VILA-U: A unified foundation model integrating visual understanding and generation[A]. 2024. arXiv: 2409.04429.

[18] CHA J, KANG W, MUN J, et al. Honeybee: Locality-enhanced projector for multimodal LLM [C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024.

[19] DUGGAL S, ISOLA P, TORRALBA A, et al. Adaptive length image tokenization via recurrent allocation[C]//Proceedings of International Conference on Learning Representations. 2025.

[20] HE K, CHEN X, XIE S, et al. Masked autoencoders are scalable vision learners[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 16000-16009.

[21] TODERICI G, VINCENT D, JOHNSTON N, et al. Full resolution image compression with recurrent neural networks[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2017: 5306-5314.

[22] VAN DEN OORD A, VINYALS O, KAVUKCUOGLU K. Neural discrete representation learning[C]//Advances in Neural Information Processing Systems: Vol. 30. 2017: 6306-6315.

[23] ESSER P, ROMBACH R, OMMER B. Taming transformers for high-resolution image synthesis[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2021: 12873-12883.

[24] RIPPEL O, GELBART M, ADAMS R. Learning ordered representations with nested dropout[C]//Proceedings of International Conference on Machine Learning. 2014: 1746-1754.

[25] KUSUPATI A, BHATT G, REGE A, et al. Matryoshka representation learning[C]//Advances in Neural Information Processing Systems: Vol. 35. 2022.

[26] BACHMANN R, ALLARDICE J, FINI E, et al. FlexTok: Resampling images into 1D token sequences of flexible length[C]//Proceedings of International Conference on Machine Learning. 2025.

[27] YAN W, ZAHARIA M, MNIH V, et al. ElasticTok: Adaptive tokenization for image and video[C]//Proceedings of International Conference on Learning Representations. 2025.

[28] DEHGHANI M, GOUWS S, VINYALS O, et al. Universal transformers[C]//Proceedings of International Conference on Learning Representations. 2019.

[29] BAI S, KOLTER J Z, KOLTUN V. Deep equilibrium models[C]//Advances in Neural Information Processing Systems: Vol. 32. 2019: 690-701.

[30] GIANNOU A, RAJPUT S, SOHN J Y, et al. Looped transformers as programmable computers [C]//Proceedings of International Conference on Machine Learning. 2023: 11398-11442.

[31] JABRI A, FLEET D, CHEN T. Scalable adaptive computation for iterative generation[C]//Proceedings of International Conference on Machine Learning. 2023.

[32] HO J, JAIN A, ABBEEL P. Denoising diffusion probabilistic models[C]//Advances in Neural Information Processing Systems: Vol. 33. 2020: 6840-6851.

[33] LEE C Y, XIE S, GALLAGHER P, et al. Deeply-supervised nets[C]//Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics. 2015: 562-570.

[34] CHANG H, ZHANG H, JIANG L, et al. MaskGIT: Masked generative image transformer[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 11315-11325.

[35] TIAN K, JIANG Y, YUAN Z, et al. Visual autoregressive modeling: Scalable image generation via next-scale prediction[C]//Advances in Neural Information Processing Systems: Vol. 37. 2024.

[36] GAO X, SHOU M Z. Diffusion autoregressive models [A]. 2025.

[37] SARAEE E, JALAL M, BETKE M. SAVOIAS: Visual complexity and aesthetics in a large dataset of paintings[A]. 2018.

xx xxx